



RISK-CRITICAL SCENARIO GENERATION FOR
LANE-CHANGING (LC) AUTONOMOUS DRIVING (AD)
TASK ON MULTI-LANE HIGHWAY, USING
MULTI-AGENT SOFT ACTOR-CRITIC
REINFORCEMENT LEARNING (MASAC-RL)

by

ALTEZA Gabriel Pascua

Department of Mechanical & Aerospace Engineering
School of Engineering

Degree thesis is submitted for

Supervisors:
Dr. Tatsuya Suzuki
Dr. Hiroyuki Okuda

Nagoya University
August, 2026

0.1 Abstract

0.2 Introduction

The deployment of autonomous driving technologies into society has been one of the most delayed advancements to the field of autonomous vehicles, especially dealing with human pushback [?]. As such, researchers and scientists have been working towards developing various methods to test the safety and robustness of autonomous driving (AD) tasks on the road.

One of the least-explored AD task in terms of safety testing is the lane-changing or lane-intrusion AD task in which the vehicle tries to insert itself within a specific gap. Past papers have employed both static methods of evaluating the ability of a vehicle’s lane changing, especially with regard to global standards, such as the ISO. However, there remains a huge gap in the available research on dynamic methods wherein obstacles are moving or actively working against the system under test (SUT). As such, this research aims to create a safety verification system for the lane-changing or lane-intrusion task into the gap of a leading-following pair of vehicles. Moreover, this research will test the lane-changing logic’s ability to withstand multiple lane changes, coming in from a merging ramp and traversing the highway to reach the preset desired exit ramp situated on the other edge of the highway.

Reinforcement learning, more broadly, machine learning, has been a hot area of research for its efficacy as a research methodology tool in solving problems that are not easily solvable through traditional white-box means. Reinforcement learning, which uses a neural network to learn from repeated simulations within a set environment, learns from its mistakes and successes in order to develop a policy that grows stronger and eventually converges to a robust neural network for the decision-making of an appropriate action to take within the environment under test.

In the field of autonomous driving safety testing, one of the most important parts to research on is the coverage of the tests, namely how diverse the testing scenarios generated are from one another in order to ensure that a wider scope of possible situations are covered. Moreover, rare cases in which risk of crashing is high are difficult to quantify and hard-code manually, while they remain to be the most important to be covered in the overall scope, especially since they are directly linked to human comfortability and safety on the road when the vehicles employing such are eventually deployed on the road.

Hence, this study aims to use specifically a multi-agent soft-actor-critic reinforcement learning framework to control a platoon of moving vehicles and let them learn how to get themselves as close as possible to inducing high risk of crash within the SUT without actually letting it crash. Moreover, the soft-actor-critic portion of this framework aims to maximize the entropy of the discovery of appropriate actions. Rather than outputting one proven action that works, the policy outputs a probability mapping of various non-clusterable actions that all efficiently maximize risk of crash in the SUT. In such way, the coverage of test-case generation is hypothetically increased, and may be evaluated against previous baseline methods for its strength in generating high-risk edge cases for lane-changing and lane-intrusion scenarios on a highway.

0.3 Theory

0.3.1 Program Architecture

The architecture of the file tree is as follows:

- Global planning of lane-changing SUT
- Safety corridor generation for use of lane-changing SUT
- Longitudinal movement MPC
- Lateral movement MPC
- Platoon MASAC-RL

The lane-changing SUT will be governed by a global planner which includes information on the target position and calculation of urgency for making lane changes (i.e., increases steadily the closer to the exit the SUT gets). In the lane-changing SUT's calculation of trajectory and path-planning as well, it must determine the "safety corridor" or maximum area for plausible intrusion before performing the lane change.

On the other hand, the platoon will be using the previously mentioned MASAC-RL framework in order to instruct where the adversary must head in the next step. The neural network will be constantly under improvement until convergence, the limit of which is to be defined in a later section.

Both the SUT vehicle and each of the platoon vehicles will be using two model predictive control (MPC) that govern the low-level control needed to drive the vehicles to the desired positions. The maneuver is decoupled, hence divided into two parts, namely longitudinal (along the highway) and lateral (across the highway).

0.3.2 Kinematic Bicycle Model

The vehicles all will be modeled using the kinematic bicycle model (KBM). Based on the KBM model, the states and inputs to be considered are as follows.

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \\ v_{t+1} \\ \gamma_{t+1} \end{bmatrix} = \begin{bmatrix} x_t + \Delta t v_t \cos \theta_t \\ y_t + \Delta t v_t \sin \theta_t \\ \theta_t + \Delta t \frac{v_t}{L_b} \tan(\gamma_t) \\ v_t + \Delta t a_t \\ \gamma_t + \Delta t \omega_t \end{bmatrix} \quad (1)$$

0.3.3 Lane-Changing Vehicle

The pipeline of the lane-change maneuver of the ego vehicle involves the separation of the "if," "when," and "how" frameworks of the maneuver, as developed in [?]. The "if" pertains to determining whether or not a lane-change maneuver into the next lane is appropriate. The "when" part aims to determine which gap to intrude

into and when to begin the move. Lastly, the "how" pertains to the usage of low-level controllers to execute the maneuver according to the determined appropriate gap.

LC Global Planner

In the LC, the global planner takes care of the "if" part by utilizing lane-utility values that correspond to appropriateness of a move into the said lane. The four utility values in this research are as follows:

1. Average travel time in the lane
2. Average time gap density in the lane
3. Remaining travel time in the lane
4. Urgency of reaching the lane in achieving end position

The four equations representing the above normalized lane-utility values $U_{\text{avetravtime}}$, $U_{\text{avetimegap}}$, $U_{\text{remtravtime}}$, U_{urgency} are detailed as follows:

$$d_{\max} = \beta v_{\text{ref}} \quad (2)$$

$$U_{\text{avetravtime}} = \frac{-\left| \frac{d_{\max}}{v_{\text{ref}}} - \frac{d_{\max}}{\max(\gamma, v_{\mu})} \right|}{\left| \frac{d_{\max}}{v_{\text{ref}}} - \frac{d_{\max}}{\gamma} \right|} \quad (3)$$

$$U_{\text{avetimegap}} = \frac{\min(\alpha t_{\text{gap,desired}}, t_{\text{gap},\mu})}{\alpha t_{\text{gap,desired}}} \quad (4)$$

$$U_{\text{remtravtime}} = \frac{\frac{\min(d_{\max}, d_{\text{end}})}{v_{\text{ref}}}}{\frac{d_{\max}}{v_{\text{ref}}}} \quad (5)$$

$$U_{\text{urgency}} = \exp \left(-\kappa \frac{i_{\text{current}} + 1}{n_{\text{lanes}} + 1} \cdot \Delta s \right) \quad (6)$$

The four utility values are then consolidated as seen in Equation (7).

$$U_{\text{total}} = \sum_{i \in A} w_i U_i \quad (7)$$

$$A = \{\text{avetravtime}, \text{avetimegap}, \text{remtravtime}, \text{urgency}\} \quad (8)$$

The lane with the highest utility values is then chosen as the target/desired lane, as seen in Equation (9).

$$L_{\text{desired}} = \operatorname{argmax}_{l \in L} (U_l - (1 + \zeta |l|) |U_0|) \quad (9)$$

The global planner is then made to choose the best inter-vehicle traffic gap in the desired lane and the time instance in which it shall enact the lane intrusion. To achieve such, a "time-position"-area, is formulated, corresponding to the intersection between the reachable set of the ego car given its dynamical limitations (current

velocity, allowed acceleration values, etc.), and the predicted safe lane change region, which is defined as the space that isn't occupied by any vehicle currently and across the prediction horizon.

The "time-position"-areas across the discretized horizon are summed and accumulate into a final "gap utility" value, displayed as A_g . The gap with the largest gap utility value, as seen in Equation (10), is chosen.

$$A_g = \sum_{k=0}^K \max \left(0, \min \left(x_{E_k}^{\max}, \min_{i \in F_g} (x_{i_k} - [t_{\text{gap}, \min} \min(v_{x_{\max}}, v_{x_{i_k}}) + d_s]) \right) - \max \left(x_{E_k}^{\min}, \max_{i \in B_g} (x_{i_k} + [t_{\text{gap}, \min} v_{x_{i_k}} + d_s]) \right) \right) \quad (10)$$

- $t_{\text{gap}, \min}$ is the minimum allowed time gap to the chosen vehicle
- $v_{x_{\max}}$ is the maximum allowed velocity for the ego
- x_{i_k} and $v_{x_{i_k}}$ refer to the predicted position and velocity values of the given vehicle, respectively
- d_s is a safe-distance buffer
- $x_{E_k}^{\max}$ and $x_{E_k}^{\min}$ are the highest and lowest reachable longitudinal distances for the ego, respectively

The minimum time (discretized) that it takes for a lane change maneuver must also be accounted for. and must require that the amount of discretized time that the gap utility value exists is greater than or equal to the said minimum discretized time, as seen in Equation (11)

$$k_{A_g}^{\text{end}} - k_{A_g}^{\text{start}} \geq k_{\min} \quad (11)$$

Moreover, the time instance at which the move is initiated is chosen by taking the minimum magnitude of the required control signals (e.g., acceleration/deceleration to reach the "time-position"-area starting from k_{Peri} .

$$k_{A_g}^{\text{start}} \leq k^{\text{Peri}} \leq K - k_{\min} \quad (12)$$

The critic network aims to critique the action that an entity takes. This is done by taking a set of data per iteration under review (initial state, action taken, reward received, and final state), taking the current state and action taken, running them through the critic network to receive an evaluatory Q value (i.e., experimental value), which is then compared with the Bellman-equation-derived Q value (i.e., theoretical value), calculated using the final states. The loss (i.e., error) is then calculated and used to determine how the weights in the neural network should be altered (via backpropagation).

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') \quad (13)$$

Now, this process is done over a sample of iterations, not just a singular iteration under review.

$$Target = Reward + \gamma Critic(NextState's, Actor(NextState's)) \quad (14)$$

The total reward R_t at time step t is formulated as the sum of four distinct components representing Time-to-Collision (TTC) proximity pressure, goal node achievement, ego vehicle crash penalties, and adversarial vehicle crash penalties:

$$R_t = R_{TTC} + R_{goal} + R_{ego_crash} + R_{adv_crash} \quad (15)$$

1. Time-to-Collision (TTC) Squeeze Component

Let A be the set of all controlled adversarial vehicles. For each adversary $i \in A$, the Time-to-Collision based on polygon geometry is denoted as τ_i . The proximity reward term is calculated via the following piecewise function:

$$R_{TTC} = \sum_{i \in A} f(\tau_i) \quad (16)$$

where

$$f(\tau_i) = \begin{cases} \frac{4.0}{\tau_i + 0.1} & \text{if } 1.0 \leq \tau_i \leq 4.0 \\ -10.0 & \text{if } 0.0 \leq \tau_i < 1.0 \\ 0.0 & \text{otherwise} \end{cases} \quad (17)$$

2. Goal Achievement Component

Let L_{ego} represent the current structural lane index pair (u, v) of the ego vehicle within the road network, and let $C_{ego} \in \{0, 1\}$ be a binary indicator denoting whether the ego vehicle has crashed (1 for crashed, 0 for safe):

$$R_{goal} = \begin{cases} +50.0 & \text{if } L_{ego} = ('l', 'm') \text{ and } C_{ego} = 0 \\ 0.0 & \text{otherwise} \end{cases} \quad (18)$$

3. Crash Guardrail Penalties

The severe negative constraints applied to prevent collision failures for both the ego vehicle (R_{ego_crash}) and individual adversarial vehicles (R_{adv_crash} where $C_i \in \{0, 1\}$ marks the crash state of adversary i) are defined as:

$$R_{ego_crash} = \begin{cases} -100.0 & \text{if } C_{ego} = 1 \\ 0.0 & \text{otherwise} \end{cases} \quad (19)$$

$$R_{adv_crash} = \sum_{i \in A} g(C_i) \quad \text{where} \quad g(C_i) = \begin{cases} -50.0 & \text{if } C_i = 1 \\ 0.0 & \text{otherwise} \end{cases} \quad (20)$$

4. Unified Dense Form

Expressed compactly utilizing indicator function notation $\mathbb{I}(\cdot)$, the comprehensive multi-agent reward function evaluates to:

$$R_t = \sum_{i \in A} \left[\mathbb{I}(1 \leq \tau_i \leq 4) \frac{4}{\tau_i + 0.1} - 10 \cdot \mathbb{I}(0 \leq \tau_i < 1) - 50 \cdot C_i \right] + 50 \cdot \mathbb{I}(L_{\text{ego}} = (l', m'))(1 - C_{\text{ego}}) - 100 \cdot C_{\text{ego}} \quad (21)$$

For the reinforcement learning part, a multi-agent SAC RL is used

So essentially what i wanna do is - use SAC RL to explore local extrema to look for specific coordinations - once converged enough(the actor loss, qf loss, and alpha), freeze the policy and use nondeterministic (probabilistic) to still output actions - remember that the alpha has both mean and standard dev

the more you train the policies, the higher poss that standard dev gets smaller and smaller and essentially becomes deterministic (not the best for farming edge cases of same coordination style, different configuration)

can write script that recognizes when a policy has converged enough for it to find a specific coordination and then find s

grab the